

第 6 章：The Dynamic Typing Interlude (动态类型插曲)

原书：《Learning Python》(6th Edition)，作者 Mark Lutz 本章地位：全书的枢纽章节。前 5 章给了你数据类型的"零件"，本章给你"部件之间的连接规则"——变量、对象、引用三者如何通过动态类型 (dynamic typing) 模型组织起来。理解本章，你就打通了后续所有章节：函数传参、类属性、模块导入，本质上都共用这同一个赋值模型。它也解释了困扰无数初学者的"为什么列表会自己变""为什么 is 和 == 结果不一样"。

6.1 开篇引言

英文原文

In the prior chapter, we began exploring Python's core object types in depth by studying Python numbers and their operations. We'll resume our object type tour in the next chapter, but before we move on, it's important that you get a handle on what may be the most fundamental idea in Python programming and is certainly the basis of much of both the conciseness a

nd flexibility of the Python language: dynamic typing, and the polymorphism it implies. As you'll see here and throughout this book, in Python, we do not need to declare the specific types of the objects our scripts use. In fact, most programs should not care about specific types—on purpose. By avoiding constraints this way, code naturally works in many contexts and often more than expected. Because dynamic typing is the root of this flexibility, and is also a potential stumbling block for newcomers, let's take a brief side trip to explore the model here. At the end of the trip, we'll also make a short stop at the paradox of type hinting, to learn why you should avoid it.

中文翻译

前一章，我们通过研究 Python 的数字对象及其操作，开始深入探索 Python 的核心对象类型。我们将在下一章继续对象类型之旅，但在继续之前，你必须先掌握 Python 编程中也许最根本的思想——它无疑也是 Python 语言许多简洁性与灵活性赖以存在的基础：那就是动态类型，以及它隐含的多态（polymorphism）。正如你将在本书此处及全书中看到的，在 Python 中，我们不需要声明脚本所用的对象的具体类型。事实上，大多数程序故意不应该关心具体类型。通过这种方式避免约束，代码自然能在许多上下文中工作，而且常常超出预期。由于动态类型是这种灵活性的根源，同时也是新手的潜在绊脚石，让我们花点时间绕个小路，深入探索这个模型。在旅途终点，我们还会在“类型标注（type hinting）的悖论”处稍作停留，了解为什么你应该避开它。

深度理解

·核心概念：本章名字里的"Interlude（插曲）"很妙——它不是继续介绍新对象类型，而是暂时停下脚步，把前面代码里已经在发生的事讲清楚。核心就一句话：类型信息存放在对象身上，而不是变量身上，所以变量可以"随时换对象"，而代码却照常工作。

·"on purpose（故意的）"是关键词：大多数人以为"不声明类型"是 Python 的缺陷或偷懒，Lutz 明确告诉你这是刻意设计。不约束类型 → 代码自动适应很多场景 → 这就是多态的根源。

·为什么它是绊脚石：C/C++/Java 出身的程序员带着"变量有类型"的思维定势，面对 `a = 3; a = "hack"` 这种代码会极度不适。本章就是要把这个思维定势拧过来。

·生活例子：想象一个"送货地址"的概念。静态类型语言像"固定信箱"——信封上印死了只能装某家公司的信；Python 的变量则像"可张贴的标签"——标签贴在快递盒子（对象）上，盒子换了，标签撕下来重贴就行，标签本身没有类型。

6.2 缺失的声明语句（The Case of the Missing Declaration Statements）

英文原文

If you have a background in statically typed language

s like C, C++, or Java, you might find yourself a bit perplexed at this point in the book. So far, we've been using variables without declaring their existence or their types, and it somehow works. When we type `a = 3` in an interactive session or program file, for instance, how does Python know that `a` should stand for an integer? For that matter, how does Python know what `a` is at all? Once you start asking such questions, you've crossed over into the domain of Python's dynamic typing model. In Python, types are determined automatically at runtime ("dynamically"), not in response to declarations added to code ahead of time ("statically"). This means that you never need to declare variables—a concept that may be simpler to grasp if you keep in mind that it all boils down to variables, objects, and the links between them, as the next section explains.

中文翻译

如果你有 C、C++ 或 Java 这类静态类型语言 (statically typed language) 的背景知识, 此刻读到这里可能会感到相当困惑。到目前为止, 我们一直在使用变量, 却从未声明它们的存在或类型, 而它居然能正常工作。例如, 当我们在交互式会话或程序文件中输入 `a = 3` 时, Python 如何知道 `a` 应该代表一个整数? 更进一步, Python 到底如何知道 `a` 是什么? 一旦你开始问这样的问题, 你就已经踏入了 Python 动态类型模型的领域。在 Python 中, 类型是在运行时 (runtime) 自动确定的, 也就是"动态"的, 而不是响应于预

先写在代码里的声明（"静态"的）。这意味着你永远不需要声明变量——如果你牢记这一切最终归结为变量、对象以及它们之间的链接，这个概念就更容易理解了，下一节将对此进行解释。

深度理解

·核心概念：静态类型（static typing）vs 动态类型（dynamic typing）。静态 = 编译期就由你写死类型；动态 = 运行时由解释器看对象"自己报"类型。

·底层视角：`a = 3` 没有"声明"，Python 是怎么运行的？答案是：压根不需要"知道的步骤"。Python 先把字面量 `3` 编译成"加载整型常量"的字节码指令，执行时在内存分配一个 `int` 对象，再把名字 `a` 指向它。名字查找是运行时的事，与类型无关。

·设计原因：把"变量是否有类型"这个包袱删掉，换来函数、类、模块全部通用化——一个打印函数能打印世界上任何对象，因为只要传进对象，类型自然从对象上来。

·初学者误区：很多人问"Python 怎么知道 `a` 是整数？"——答案不是"它判断出来了"，而是"它根本没去判断"。`a` 没有类型，它只是指向那个"知道自己是 `int`"的对象。问"`a` 是什么类型"就像问"这根绳子是什么材料"——绳子没有材料，绳子上拴着的东西才有。

·生活例子：你在公司打印文件，"打印"这个动作不关心文件是 Word 还是 PDF（类型归文件自己管），只要它是可打印的对象就能打。这就是动态类型的日常体验。

6.3 变量、对象与引用 (Variables, Objects, and References)

英文原文

As you've seen in many of the examples used so far in this book, when you run an assignment statement such as `a = 3` in Python, it works even if you've never told Python to use the name `a` as a variable, or that `a` should stand for an integer-type object. In the Python language, this all pans out in a very natural way, as follows:

Variable creation: A variable (also known in Python as a name), like `a`, is created when your code first assigns it a value. Future assignments change the value of the already created name. Technically, Python detects some names before your code runs (e.g., locals in functions), but you can think of it as though initial assignments make variables.

Variable types: A variable itself never has any type information or constraints associated with it. In Python, the notion of type lives with objects, not names. Variables are generic in nature; they always simply refer to a particular object at a particular point in time.

Variable use: When a variable appears in an expression, it is immediately replaced with the object that it currently refers to, whatever that may be. Further, all variables must be explicitly assigned before they can be used; referencing unsigned variables results in errors. In sum, variables are

are created when assigned, can reference any type of object, and must be assigned before they are referenced. This means that you never need to declare names used by your script, but you must initialize names before you can update them; counters, for example, must be initialized to zero before you can add to them. This dynamic typing model is strikingly different from the typing model of traditional languages. When you are first starting out, the model is usually easier to understand if you keep clear the distinction between names and objects. For example, when we say this to assign a variable a value in a Python REPL or script:

```
python >>> a = 3 # Assign a name to an object at least
```

conceptually, Python will perform three distinct steps to carry out the request. These steps reflect the operation of all assignments in the Python language: 1. Create an object to represent the value 3. 2. Create the variable a, if it does not yet exist. 3. Link the variable a to the new object 3. The net result will be a structure inside Python that resembles Figure 6-1. As sketched, variables and objects are stored in different parts of memory and are associated by links (the link is shown as a pointer in the figure). Variables always link to objects and never to other variables, but larger objects may link to other objects (for instance, a list object has links to the objects it contains). Figure 6-1. Names (a.k.a. variables) and objects after running the assignment `a = 3` These links from variables to objects are

called references in Python—a kind of association, implemented as an object's address in memory. Whenever variables are later used (i.e., referenced), Python automatically follows the variable-to-object links. This is all simpler than the terminology may imply. In concrete terms: Variables are named entries in a system table, with spaces for links to objects. Objects are pieces of allocated memory, with enough space to represent the values for which they stand. References are automatically followed pointers from variables to objects. At least conceptually, each time you generate a new value in your script by running an expression, Python creates a new object (i.e., a chunk of memory) to represent that value. As an optimization, Python internally caches and reuses certain kinds of unchangeable objects, such as small integers and strings (each 0 is not really a new piece of memory—more on this caching behavior later). But from a logical perspective, it works as though each expression's result value is a distinct object and each object is a distinct piece of memory. Technically speaking, objects have more structure than just enough space to represent their values. Each object also has two standard header fields: a type designator used to mark the type of the object, and a reference counter used to determine when it's OK to reclaim the object. To understand how these two header fields factor into the model, we need to move on.

中文翻译

正如你在本书迄今许多例子中所见，当你在 Python 中运行 `a = 3` 这样的赋值语句时，即使你从未告诉 Python 把名字 `a` 用作变量，也没说 `a` 应该代表一个整数类型的对象，它依然正常工作。在 Python 语言里，这一切以一种非常自然的方式运转，具体如下：

变量的创建：变量（在 Python 中也叫名字，`name`），比如 `a`，在你的代码第一次给它赋值时被创建。后续的赋值会改变这个已创建名字的值。严格来说，Python 会在代码运行前就侦测到一些名字（例如函数中的局部变量），但你可以把它想象成“首次赋值创建了变量”。

变量的类型：变量本身从不带任何类型信息或约束。在 Python 中，类型住在对象上，而不是名字上。变量本质上是通用的；它们始终只是“在某个特定时刻引用某个特定对象”。

变量的使用：当一个变量出现在表达式中时，它会被立即替换成它当前所引用（`refer to`）的对象，无论那是什么对象。此外，所有变量在使用前必须被显式赋值；引用未赋值的变量会导致错误。总而言之：变量在赋值时被创建，可以引用任何类型的对象，并且必须先赋值才能被引用。这意味着你永远不需要声明脚本所使用的名字，但你必须先初始化名字才能再更新它；例如计数器必须先初始化为零，才能往上累加。这种动态类型模型与传统语言的类型模型有着惊人的差异。初学时，如果你始终分清“名字”与“对象”的区别，这个模型通常更容易理解。例如，当我们在 Python 的 REPL 或脚本中说下面这句话来给变量赋值时：

```
python >>> a = 3 # 给名字 a 赋一个对象至少在概念上，Python 会分
```

三个清晰的步骤来完成这个请求。这三个步骤反映了 Python 语言中所有赋值的运作方式：

1. 创建一个对象来代表值 3

。2. 创建变量 `a` (如果它尚不存在)。3. 把变量 `a` 链接到新对象 3。最终结果会是 Python 内部类似图 6-1 的一种结构。如图所示, 变量和对象存放在内存的不同区域, 通过"链接"关联 (图中用指针表示链接)。变量总是链接到对象, 从不链接到另一个变量; 但较大的对象可以链接到其他对象 (例如, 列表对象持有指向它所包含对象的链接)。图 6-1: 执行赋值 `a = 3` 之后的名字 (又名变量) 和对象这些从变量到对象的链接, 在 Python 中被称为引用 (reference) ——一种关联方式, 在实现上就是对象在内存中的地址。每当变量之后被使用 (即被引用) 时, Python 会自动沿着"变量→对象"的链接走过去。这一切比术语给人的感觉更简单。具体来说: 变量: 系统表格中的具名条目, 带有存放对象链接的空间。对象: 被分配出来的一块内存 (chunk of memory), 有足够的空间来代表它们所代表的值。引用: 从变量自动"顺着走过去"指向对象的指针。至少在概念上, 每次你在脚本中运行一个表达式产生一个新值时, Python 都会创建一个新对象 (即一块内存) 来代表这个值。作为一种优化, Python 在内部会缓存并复用某些不可变对象, 例如小整数和字符串 (每个 0 并不是真正新分配的一块内存——稍后会更多讨论这种缓存行为)。但从逻辑视角看, 一切就好像"每个表达式的结果值都是一个独立的对象, 每个对象都是一块独立的内存"。严格从技术上讲, 对象的结构比"刚好够存放它们的值"要更复杂。每个对象还有两个标准的头字段 (header field): 一个类型指示符 (type designator), 用来标记对象的类型; 一个引用计数器 (reference counter), 用来判断何时可以回收这个对象。要理解这两个头字段如何融入这个模型, 我们需要继续往下走。

深度理解

·核心概念（三条黄金定律）：① 变量在首次赋值时创建；② 变量本身无类型，类型属于对象；③ 变量必须先赋值才能引用。以后到第 9 章学函数、第 28 章学类，这三条依然成立——全语言只有一个赋值模型。

·底层实现——对象三要素：书里说的两个"头字段"是 CPython 的 PyObject 结构，但我们可以把概念扩展成对象三要素这个贯穿全书的心法：

·value（值/内容）：对象实际装的数据，如数字 3、字符串 "hack"。

·type（类型）：对象的类型指示符，决定你能对它做什么操作（能不能加、能不能改）。

·id（身份/地址）：对象在内存的位置，CPython 中约等于地址，可用内置函数 id() 查询。

·三者都是对象的属性，与变量无关。判断两个对象"内容一样"用 ==（比 value），"是同一个对象"用 is（比 id）。

·为什么这样设计：把类型挂在对象上，对象就"自报家门"——任何函数拿到任何对象都能问"你是什么类型、能做什么"。这正是多态的底层支撑，也让 Python 无需编译期类型检查。

·实际问题：三要素也解释了最常见的 bug 来源——两个变量"看起来一样"（value 相同）却不是同一个东西（id 不同），对可变对象原地修改时就会"一个改、处处变"。

·初学者误区：

·以为"变量就是盒子，赋值就是往盒子里塞东西"。不是一—变量是"贴纸/标签"，赋值是把标签贴到对象上；再赋值是撕下来贴到新对象上，旧对象没人贴就等着被回收。

·以为"声明了类型"——`a: int` 只是标注，不创建变量也不强制类型（后面 6.10 节专讲）。

·以为 0 每次都新建——小整数和短字符串会被缓存复用，`0 is 0` 都是 `True`。

代码分析

```
>>> a = 3 # a 3
```

·解释器如何处理：① 编译期：源码被编译成字节码，3 作为一个常量被放进"常量表"（`coconsts`），对应指令大概是 `LOADCONST 3` 后面跟 `STORENAME'a'`。② 运行期：执行 `LOADCONST` 时从常量表取出 `int` 对象（首次可能是新建或复用缓存），执行 `STORENAME'a'` 时在"名字→对象"的映射表中登记 `a → 该对象地址`。整个过程没有一步在做"类型判断"——类型在对象创建那一刻就由类型指示符决定了。

·三个概念步骤对应：`LOADCONST 3` $\approx$ 步骤 1（拿对象），`STORENAME a` $\approx$ 步骤 2 + 3（建名字 + 建链接）。字节码指令设计就是这三步思想的最直观体现。

·设计思想：赋值 = "让名字引用对象"，不同时期学到的元组解包、链式赋值、函数实参传值，全都在复刻这一条

。

技术侧栏：了解 Python 引用（写给 C 程序员）—— PYTHON REFERENCES FOR C PROGRAMMERS

英文原文

Readers with a background in C may find Python references similar to C pointers (i.e., memory addresses). In fact, references are implemented as pointers by CPython internally, and they often serve the same roles, especially with objects that can be changed in place (more on this later). Because references are always automatically dereferenced when used, though, you can never actually do anything useful with a reference itself. As noted ahead, the referenced object's address may be returned by the `id` built-in as a unique ID, but even this isn't guaranteed: see Python's manuals. This lack of pointers avoids an entire category of notorious C bugs. But you can think of Python references as C "void" pointers that are automatically followed when used, without going too far off base.

中文翻译

有 C 语言背景的读者会发现，Python 的引用类似于 C 的指针（pointer，即内存地址）。事实上，在 CPython 内部，引用就是用指针实现的，而且它们常常扮演相同的角色，尤其是对于那些可以原地修改（in-place change）的对象（稍后详述）。不过，因为引用在使用时总是被自动解引用（dereference），所以你实际上永远无法对引用本身做

什么有用的事。如前所述，被引用对象的地址可以通过内置函数 `id` 以唯一 ID 的形式返回，但连这一点也不保证：参见 Python 官方手册。“没有指针”这件事，避免了整整一个类别的著名 C 语言 bug（如悬垂指针、缓冲区越界、空指针解引用）。你可以把 Python 的引用理解成一种“使用时自动被追寻的 C void 指针”，这不会错得太离谱。

深度理解

- 核心概念：Python 引用 $\approx$ 自动解引用的指针。你只能“用它所指向的对象”，不能操纵指针本身做加减（`ptr = ptr + 1` 这种 C 玩法在 Python 不存在）。
- 底层实现：CPython 中每个变量实际上是“名字表”里的一个指针 `PyObject`，指向堆上的对象。你写代码时永远碰不到这个指针值本身。
- 设计原因：C 的指针 bug（内存越界、悬垂引用、双重释放）是人类错误的大杀器。Python 把“自动跟随”和“引用计数回收”打包进语言底层，直接消灭这类 bug 的土壤。
- 实际问题：如果你听说“Python 变量都是指针”，要立刻警惕 C 思维的两个误导：① C 里改一个指针指向不影响原对象——这确实也对；② 但 C 里“指针本体地址不变”不等于“对象不变”——Python 的原地修改（如 `L[0]=x`）恰恰是顺着引用去改目标对象的内部。这提醒我们：Python 引用是“到对象的引用”，不是“到变量的引用”。
- 初学者误区：`id()` 返回的是“唯一标识”，作者特别提醒别把它当内存地址用——CPython 里通常是地址，但语言规范不保证，其他实现（Jython/PyPy）就完全不同。

6.4 类型存在于对象中，而非变量中 (Types Live with Objects, Not Variables)

英文原文

To see how object types come into play, watch what happens if we assign a variable multiple times: `python >>> a = 3 # It's an integer >>> a = 'hack' # Now it's a string >>> a = 1.23 # Now it's a floating point` This isn't typical Python code, but it does work—a starts out as an integer, then becomes a string, and finally becomes a floating-point number. This example tends to look especially odd to ex-C programmers, as it appears as though the type of `a` changes from integer to string when we say `a = 'hack'`. However, that's not really what's happening. In Python, things work more simply. Names have no types; as stated earlier, types live with objects, not names. In the preceding listing, we've simply changed `a` to reference different objects. Because variables have no type, we haven't actually changed the type of the variable `a`; we've simply made the variable reference a different type of object. In fact, again, all we can ever say about a variable in Python is that it references a particular object at a particular point in time. Objects, on the other hand, know what type they are—each object contains a header field that tags the object with its type. The integer obj

ect 3, for example, will contain the value 3, plus a designator that tells Python that the object is an integer (strictly speaking, a pointer to an object called `int`, the name of the integer type). The type designator of the `'hack'` string object points to the string type (called `str`) instead, and `1.23` points to `float`. Because objects know their types, variables don't have to. To recap, types are associated with objects in Python, not with variables. In typical code, a given variable usually will reference just one kind of object. Because this isn't a requirement, though, you'll find that Python code tends to be much more flexible than you may be accustomed to—if you use Python well, your code might work on many types automatically. As mentioned, objects have two header fields, a type designator and a reference counter. To understand the latter of these, we need to move on and take a brief look at what happens at the end of an object's life.

中文翻译

要看看对象类型是如何起作用的，注意如果我们对同一个变量多次赋值会发生什么：`python >>> a = 3 # 这是一个整数`
`>>> a = 'hack' # 现在它是一个字符串`
`>>> a = 1.23 # 现在它是一个浮点数`
这虽然不是典型的 Python 代码，但它是可以运行的——`a` 从整数开始，然后变成字符串，最后变成浮点数。这个例子在曾经的 C 程序员看来格外怪异，因为当我们说 `a = 'hack'` 时，看起来仿佛 `a` 的类型从整数变成了字符串。然而，真正发生的并非如此。在 Python 中，

事情要简单得多。名字没有类型；如前所述，类型存在于对象中，而不是名字中。在上面这段代码里，我们只是让 `a` 改去引用不同的对象而已。因为变量没有类型，我们并没有真正改变变量 `a` 的"类型"；我们只是让这个变量引用了一个不同类型的对象。事实上，再说一遍，在 Python 中我们对一个变量所能说的全部内容，只有"它在某个特定时刻引用某个特定对象"。而对象则知道自己是什么类型——每个对象都含有一个把它"贴标签"标记类型的头字段。例如整数对象 `3`，会包含值 `3`，外加一个指示符，告诉 Python 这个对象是整数（严格说，是一个指向名为 `int` 的对象的指针，`int` 就是整数类型的名字）。'hack' 字符串对象的类型指示符指向字符串类型（叫 `str`），而 `1.23` 指向 `float`。因为对象知道自己的类型，变量就不必知道了。总结一下：在 Python 中，类型是与对象关联的，而不是与变量关联的。在典型代码中，某个给定变量通常只会引用一种对象。但因为这并非强制要求，你会发现 Python 代码往往比你习惯的灵活得多——如果你把 Python 用得好，你的代码可以自动适用于许多类型。如前所述，对象有两个头字段：类型指示符和引用计数器。要理解后者，我们需要继续前进，简要看看一个对象生命尽头会发生什么。

深度理解

·核心概念：这是动态类型最核心的一句话——"类型住在对象里，不住在名字里"。变量字典里只有"名字 → 对象"的键值对，没有"类型"这一列。

·底层实现：CPython 里每个对象开头是一个 `PyObjectHeader` 头：`obrefcnt`（引用计数）+ `obtype`（指向类型对象的指针）。`type(a)` 就是沿着 `obtype` 指针找到"类型对

象"再取其名字；int、str、float 在 Python 里本身就是"t
ype 类型的对象"。这就是为什么 type(int) 是 type——
一切皆对象，连类型也是对象。

·设计原因：如果变量有类型，那么"同一个函数接收不同
类型"就得靠重载或泛型来完成；而类型归对象，函数天然
通用。Lutz 强调"这是灵活性而不是 bug"。

·实际问题：这带来真正的多态——比如自定义一个对象只
要实现 +，就能自动参与加法运算；Python 的"鸭子类型
"（第 32 章会展开）就是本节思想的自然延伸。

·初学者误区：ex-C 程序员受"变量有类型"困扰最深，他
们会问"那 Python 怎么不管类型错误？"——因为根本没
错误可言：a = 'hack' 不是"类型变了"的错误，而是"合法
的换了个引用对象"。类型检查的职责被推迟到了真正的操
作发生那一刻（比如把字符串和数字相加才会在运行时报
TypeError）。

6.5 对象会被垃圾回收 (Objects Are Garbage -Collected)

英文原文

In the prior section's listings, we assigned the variable a to different types of objects in each assignment. But when we reassign a variable, what happens to the value it was previously referencing? For example, after the following statements, what happens to the ob

ject 3? python >>> a = 3 >>> a = 'text' The answer is that in Python, whenever a name is assigned to a new object, the space held by the prior object is reclaimed if it is not referenced by any other name or object. This automatic reclamation of objects' space is known as garbage collection and makes life much simpler for programmers of languages like Python that support it. To illustrate, consider the following example, which sets the name x to a different object on each assignment: python >>> x = 99 # Reclaim 99 now (unless referenced elsewhere) >>> x = 'Python' # Reclaim 'Python' now (ditto) >>> x = 3.1415 # Reclaim 3.1415 now (ditto) >>> x = [1, 2, 3] # Reclaim 3.1415 now (ditto) First, notice that x is set to a different type of object each time. Again, the effect is as though the type of x is changing over time, but this is not really the case. Remember, in Python types live with objects, not names. Because names are just generic references to objects, this sort of code works naturally. Second, notice that references to objects are discarded along the way. Each time x is assigned to a new object, Python reclaims the prior object's space. For instance, when it is assigned the string 'Python', the object 99 is immediately reclaimed (assuming it is not referenced anywhere else)—that is, the object's space is automatically thrown back into the free space pool, to be reused for a future object. Internally, Python accomplishes this feat by keeping a counter in every object that keeps t

rack of the number of references currently pointing to that object. As soon as—and exactly when—this counter drops to zero, the object's memory space is automatically reclaimed. In the preceding listing, we're assuming that each time `x` is assigned to a new object, the prior object's reference counter drops to zero, causing it to be reclaimed. The most immediately tangible benefit of garbage collection is that it means you can use objects liberally without ever needing to allocate or free up space in your script. Python will make objects clean up their unused space for you as your program runs. In practice, this eliminates a substantial amount of bookkeeping code required in lower-level languages such as C and C++.

中文翻译

上一小节的代码里，我们在每次赋值时把变量 `a` 指向不同类型的对象。但当我们重新给变量赋值时，它之前引用的那个值会怎样？例如，执行下列语句之后，对象 3 会怎样？
`python >>> a = 3 >>> a = 'text'` 答案是：在 Python 中，每当一个名字被赋给一个新对象时，如果之前的对象没有被任何其他名字或对象所引用，那么它所占据的空间就会被回收。这种自动回收对象空间的机制被称为垃圾回收（garbage collection），它让支持此机制的语言（如 Python）的程序员生活轻松得多。为了说明这一点，请看下面的例子：它每次赋值都把名字 `x` 指向一个不同的对象：
`python >>> x = 99 # 现在回收 99（除非它还被别处引用）>>> x = 'Python' # 现在回收'Python'（同上）>>> x = 3.1415 #`

现在回收 3.1415 (同上) >>> x = [1, 2, 3] # 现在回收 3.1415 (同上) 首先注意, x 每次都被设置为不同类型的对象。又一次, 效果看起来仿佛 x 的类型随时间在变化, 但实际情况并非如此。记住, 在 Python 中类型跟随对象, 而非名字。因为名字只是对对象的通用引用, 这类代码自然可以工作。其次注意, 沿途会有引用被丢弃。每次 x 被赋给一个新对象, Python 就会回收之前对象的空间。例如, 当 x 被赋为字符串'Python' 时, 对象 99 会立刻被回收(假设它没有在任何其他地方被引用)——也就是说, 这个对象的空间会被自动丢回空闲空间池 (free space pool), 供将来的对象复用。在内部, Python 是靠给每个对象维护一个计数器来完成这项壮举的, 这个计数器记录当前有多少个引用正指向该对象。当这个计数器降到零的那一刻——不多不少恰好在那一刻——对象的内存空间就会被自动回收。在上面的代码里, 我们假设每次 x 被赋给新对象时, 之前对象的引用计数都降到了零, 从而触发了回收。垃圾回收最直接、最实在的好处是: 你可以尽情地使用对象, 而永远不需要在脚本里手动分配或释放空间。Python 会在程序运行时替对象清理不再使用的空间。实际上, 这省掉了 C、C++ 这类底层语言所需的大量记账 (bookkeeping) 代码。

深度理解

·核心概念: 引用计数 (reference counting) ——每个对象头部记着“现在有几个引用指着我”, 减到 0 立刻释放内存, 空间回到空闲池待复用。

·底层实现: CPython 里每个对象头有 obrefcnt。变量赋值时旧引用计数 --, 新引用 ++; refcnt 变 0 就调用该类型的“析构函数”并释放内存。并注意: 函数参数、容器元

素、局部变量都算引用，所以引用的"增"和"减"遍布解释器各处。

·设计原因：把内存管理交给语言，程序员不必考虑 `free()` / `malloc()`，避免内存泄漏和双重释放。Lutz 说这省掉了"大量记账代码"——他自己 20 年教学里最深的感慨之一。

·实际问题：3.5 一节例子顺序有个小彩蛋——赋值 `[1, 2, 3]` 时回收的是 3.1415，注释也这么写（书里原样），因为 3.1415 是上一个被解除引用的对象，而 `[1,2,3]` 是刚新建的。回收的永远是没有被引用的旧值。

·初学者误区：

1. 以为"垃圾回收 = 定时清理内存"——错，CPython 是立即计数归零、立即回收，不是等到某个时刻。"按需手动 `del`"并非必须，`del` 只是帮你主动减一个引用。

2. 以为回收能瞬间观察到——本书也提醒"缓存对象不一定真被回收"（见 6.8 节，99 很可能是"假回收真缓存"）。

3. 以为 Python 只有一个回收机制——其实还有循环引用检测器（cyclical garbage collector），因为互相引用的两个对象计数永不为 0，必须靠后代检测器发现并回收（见下方侧栏）。

代码分析

```
>>> x = 99          # 99
>>> x = 'Python'    # 'Python'
>>> x = 3.1415       # 3.1415
>>> x = [1, 2, 3]    # 3.1415
```

·解释器如何处理：以 `x = 99` 为例：执行 `STORENAME x` 时，把名字 `x` 指向新 `int` 对象 99（引用计数从 0 变 1）；`x = 'Python'` 时，`'Python'` 常量被取出、并入引用名表（计数 +1），同时旧的 99 不再被 `x` 引用（计数 -1 变 0）→ 立刻触发 99 的析构与释放。其余赋值以此类推。注意的是每一行赋值同时做“新对象引用 +1”和“旧对象引用 -1”两件事，顺序由解释器保证不产生泄漏。

·一个细节：为什么 `x = [1, 2, 3]` 前一条注释是“现在回收 99”后一条是“回收 3.1415”？因为回收动作发生在前一对象不再被引用之时；队列就是“这次赋值前我还引用着的那个对象”。可以顺带记住练习：真去 `sys.getrefcount(99)` 会看到计数不为 1，因为小整数 99 被缓存了（见 6.8）。

·设计思想：把“何时释放”的决策从程序员手里收走，改为对象自己“数自己还有没有存在价值”。这是“数据驱动”思维在内存管理上的体现：对象自己最清楚自己什么时候该走。

技术侧栏：更多关于 Python 垃圾回收——MORE ON PYTHON GARBAGE COLLECTION

英文原文

Technically speaking, Python's garbage collection is based mainly upon reference counters, as described here, but it also has a component that detects and reclaims objects with cyclic references in time. This component can be disabled if you're sure that your code doesn't create cycles, but it is enabled by default. Both

references and this optional component do garbage collection, but the latter may be what users of some other languages lacking reference counts think of as "garbage collection." Circular references are a classic issue in reference-count garbage collectors. Because references are implemented as pointers, it's possible for an object to reference itself, or reference another object that does. For example, exercise 6 in "Test Your Knowledge: Part I Exercises" and its solution in Appendix B show how to create a cycle easily by embedding a reference to a list within itself (e.g., `L.append(L)`). The same phenomenon can occur for assignments to attributes of objects created from user-defined classes. Though relatively rare, because the reference counts for such objects never drop to zero, they must be treated specially. For more details on Python's cycle detector and collector, see the documentation for the `gc` module in Python's library manual. The best takeaway here is that garbage-collection-based memory management is implemented for you in Python, by people highly skilled at the task; it works well, even for cycles. Also note that this chapter's description of Python's garbage collector is not part of the language's definition and applies to the standard implementation of Python (a.k.a. CPython) only. Chapter 2's alternative implementations such as Jython, IronPython, and PyPy may use different schemes, though the net effect in all is similar—unused space is reclaimed for you au

tomatically, if not always as immediately.

中文翻译

严格从技术上说，Python 的垃圾回收主要基于前面讲的引用计数，但它还有一个组件，能及时检测并回收具有循环引用（circular reference）的对象。如果你确信自己的代码不会产生循环引用，这个组件可以被禁用，但默认是启用的。引用计数和这个可选组件都在做垃圾回收，但后者可能才是某些“没有引用计数”的其他语言用户所理解的“垃圾回收”。循环引用是引用计数型垃圾回收器的一个经典难题。因为引用是用指针实现的，所以一个对象有可能引用它自己，或者引用另一个反过来引用它的对象。例如，“Test Your Knowledge: Part I Exercises”（第一部分自我测验）中的第 6 题及其在附录 B 中的解答，就展示了如何通过把列表的引用嵌入列表自身（例如 `L.append(L)`）轻松制造循环。当对用户自定义类的对象属性赋值时，同样可能发生这种现象。虽然相对罕见，但由于这类对象的引用计数永远不会降到零，它们必须被特殊对待。关于 Python 循环检测器与回收器的更多细节，参见 Python 库手册中 `gc` 模块的文档。这里最重要的心得是：Python 已经由这方面的顶级专家为你实现了基于垃圾回收的内存管理；它运行得很好，即便是循环引用也能处理。还要注意：本章对 Python 垃圾回收器的描述并不是语言规范的一部分，它只适用于 Python 的标准实现（即 CPython）。第 2 章介绍的其他实现，如 Jython、IronPython、PyPy，可能采用不同的方案，不过最终效果都类似——未使用的空间会被自动回收，只是不总是那么即时。

深度理解

·核心概念：Python 的 GC = 引用计数（主力）+ 分代循环检测器（补漏）。前者负责“快准狠”地立即回收，后者负责收集引用计数算不出来的“循环垃圾”。

·底层实现：循环检测器使用分代思路——对象按存活时间分代，隔一段时间对较老的对象做“环检测”，标记并回收不可达的循环团。用 `import gc; gc.collect()` 可手动触发。

·为什么需要它：`L.append(L)` 把 L 自己放进去，L 的引用计数永不 0——引用计数模型“死锁”了，非额外的检测器不可。这是经典教科书问题，每个面试官都爱问。

·实际问题：生产中的典型循环来自“父子对象的互指”（如双向链表、GUI 控件互相回调、缓存+回调 Closure 互相持有）。现代代码习惯是尽量避免循环或显式破除；但有 gc 兜底，即使忘了也一般不会永久泄漏。

·初学者误区：① 误以为 `del` 会立刻释放——`del` 只删一个名字/引用，若还有其他引用对象仍存活；② 误以为 `is` 是“判断对象内容相等”——它是判断“是不是同一个对象（同一身份）”，与“循环引用”无关但与“共享引用”强相关，下一节正题来了；③ 误以为“引用计数归零一定立即回收”——缓存对象（小整数/短字符串）是例外，它们常驻内存池。

6.6 共享引用 (Shared References)

英文原文

So far, we've explored what happens as a single variable is assigned references to objects. Now let's introduce another variable into our interaction and watch what happens to its names and objects: `python >>> a = 3 >>> b = a` Typing these two statements generates the scene captured in Figure 6-2. The second command causes Python to create the variable `b`; the variable `a` is being used and not assigned here, so it is replaced with the object it references (3), and `b` is made to reference that object. The net effect is that the variables `a` and `b` wind up referencing the same object—that is, pointing to the same chunk of memory. Figure 6-2. Names and objects after next running the assignment `b = a` This scenario in Python—with multiple names referencing the same object—is usually called a shared reference (and sometimes and perhaps more accurately, shared object). Note that the names `a` and `b` are not linked to each other directly when this happens; in fact, there is no way to ever link a variable to another variable in Python. Rather, both variables point to the same object via their references. Next, suppose we extend the session with one more statement: `python >>> a = 3 >>> b = a >>> a = 'hack'` As with all Python assignments, this statement simply makes a

new object to represent the string value 'hack' and sets a to reference this new object. It does not, however, change the value of b; b still references the original object, the integer 3. The resulting reference structure is shown in Figure 6-3. Figure 6-3. Names and objects after finally running the assignment a = 'hack' The same sort of thing would happen if we changed b to 'hack' instead—the assignment would change only b, not a. This behavior also occurs if there are no type differences at all. For example, consider these three statements: `python >>> a = 3 >>> b = a >>> a = a + 2` In this sequence, the same events transpire. Python makes the variable a reference the object 3 and makes b reference the same object as a, as in Figure 6-2; as before, the last assignment then sets a to a completely different object (in this case, the integer 5, which is the result of the + expression). It does not change b as a side effect. In fact, there is no way to ever overwrite the value of the object 3—as introduced in Chapter 4, integers are immutable and thus can never be changed in place (see why this stuff matters?). All we can ever do is make a new integer object. One way to think of this is that, unlike in some languages, variables in Python are always pointers to objects, not labels of changeable memory areas: setting a variable to a new value does not alter the original object, but rather causes the variable to reference an entirely different object. The net effect is that assignment to a varia

ble itself can impact only the single variable being assigned. When mutable objects and in-place changes enter the equation, though, the picture changes somewhat; to see how, let's move on.

中文翻译

到目前为止，我们探索的是单个变量被赋以对象引用的情形。现在让我们再引入一个变量，看看它的名字和对象会发生什么：python >>> a = 3 >>> b = a 输入这两个语句，就会产生图 6-2 所示的情景。第二条命令让 Python 创建变量 b；a 在这里是被使用而不是被赋值，所以它会被替换成它所引用的对象（3），然后 b 被设置为引用那个对象。最终效果是：变量 a 和 b 最终引用同一个对象——也就是说，指向同一块内存。图 6-2：接着执行赋值 b = a 之后的名字和对象这种局面——多个名字引用同一个对象——在 Python 中通常被称为共享引用（shared reference，有时也更准确地叫共享对象）。注意，发生这种情况时，名字 a 和 b 之间并没有直接链接；事实上，在 Python 中你永远没有办法把变量链接到变量。二者都是通过各自的引用指向同一个对象。接下来，假设我们在会话中再扩展一条语句：python >>> a = 3 >>> b = a >>> a = 'hack' 和所有 Python 赋值一样，这条语句只是创建一个新的对象来表示字符串值 'hack'，并让 a 引用这个新对象。但是，它并不会改变 b 的值；b 仍然引用原来的对象——整数 3。得到的引用结构见图 6-3。图 6-3：最后执行赋值 a = 'hack' 之后的名字和对象如果我们改成把 b 设为 'hack'，也会发生同样的事——赋值只会改变 b，不会影响 a。即使完全没有类型差异，这个行为也一样成立。例如考虑下面三条语句：pytho

`n >>> a = 3 >>> b = a >>> a = a + 2` 在这个序列里，发生的是同样的事件。Python 让变量 `a` 引用对象 3，让 `b` 引用与 `a` 相同的对象（如图 6-2 所示）；和前面一样，最后一条赋值让 `a` 指向一个完全不同的对象（本例中是整数 5，即 `+` 表达式的运算结果）。它不会作为副作用去改变 `b`。事实上，你永远无法覆盖对象 3 的值——正如第 4 章所介绍的，整数是不可变的（immutable），因此永远不能原地修改（现在你明白为什么这些东西重要了吗？）。我们对它所能做的只有“新建一个整数对象”。换个角度理解：与某些语言不同，Python 中的变量永远是指向对象的指针，而不是“一块可变内存区域的标签”：给变量设置新值并不会改动原来的对象，而是让这个变量去引用一个全新的对象。净效果是：给变量本身赋值，只会影响被赋值的这一个变量。不过，当可变对象和原地修改加入方程时，画面就有所变化了；要明白如何变化，请继续往下看。

深度理解

- 核心概念：共享引用 = 两个（或多个）名字指向同一个对象。`b = a` 不是“把 `a` 的内容复制给 `b`”，而是“让 `b` 也贴上对象 3 这张标签”。
- 底层实现：赋值 `b = a` 编译为 `COPYNAME/LOADNAME + STORENAME`，运行时把当前 `a` 所指地址复制一份登记到 `b` ——引用计数 3 的计数从 1 变为 2。
- 设计原因：共享引用让“传递”变得极廉价（复制一个指针就行），也让不可变对象（int/str/tuple）天然安全——既然没人能改对象内容，共享一万遍也无妨。
- 实际问题（关键推论）：

- int 是不可变的: $a = a + 2$ 直接"撕掉 a 的标签, 贴到新对象 5 上", b 纹丝不动。对不可变对象, 共享引用不会造成任何意外。
- 可变量是隐患来源: $L2 = L1$ 后, 对 $L1$ 的内部做原地修改, $L2$ 会"看见"变化 (下一节)。
- 一句话记忆: "对变量赋值"永远只影响这个变量; "改变共享对象的内容"才会波及所有引用者。
- 初学者误区: 第一反应" $b = a$ 之后 a 变了 b 应该不变"?——如果 a 是"被重新赋值" (如 $a = 'hack'$), b 确实不变; 如果 a 是"原地改了共享对象的内容" (下一节的 $L1[0] = 24$), b 必然跟着变。区别在于"换标签"还是"改东西"。
- 生活例子: 共享引用就像你和朋友各拿一把同一保险柜的钥匙。换保险柜 ($a = \text{new}$) = 你换了把锁, 朋友的钥匙自然打不开你的新柜子 (他手里还是旧柜子的东西); "原地修改" = 有人钻进保险柜把里面的钱换成假钞——不需要新钥匙, 谁都看得见钱变了。

6.7 共享引用与原地修改 (Shared References and In-Place Changes)

英文原文

As you'll learn more in this part's upcoming chapters, some operations do change objects in place, but they're only supported by Python's mutable types—inclu

ding lists, dictionaries, and sets. For instance, an assignment to an offset in a list actually changes the list object itself in place, rather than generating a brand-new list object. Though you must take it somewhat on faith at this point in the book, this distinction can matter much in your programs. For objects that support such in-place changes, you need to be more aware of shared references, since a change from one name may impact others. Otherwise, your objects may seem to change for no apparent reason. Given that all assignments are based on references (including function argument passing), it's a pervasive phenomenon. To illustrate, let's take another look at the list objects introduced in Chapter 4. Recall that lists, which do support in-place assignments to positions, are simply collections of other objects, coded in square brackets:

```
python >>> L1 = [2, 3, 4] >>> L2 = L1
```

L1 here is a list containing the objects 2, 3, and 4. Items inside a list are accessed by their positional offsets, so `L1[0]` refers to object 2, the first item in the list L1. Of course, lists are also objects in their own right, just like integers and strings. After running the two prior assignments, L1 and L2 reference the same shared object, just like `a` and `b` in the prior example (see Figure 6-2). Now imagine that, as before, we extend this interaction to say the following:

```
python >>> L1 = 24
```

This assignment simply sets L1 to a different object; L2 still references the original list

much as in the preceding section. If we change this statement's syntax slightly, however, it changes its effect radically:

```
python >>> L1 = [2, 3, 4] # A mutable object >>> L2 = L1 # Make a reference to the same object >>> L1[0] = 24 # An in-place change >>> L1 # L1 is different [24, 3, 4]
>>> L2 # But so is L2! [24, 3, 4]
```

Really, we haven't changed L1 itself at line three here; we've changed a component of the object that L1 references. This sort of change overwrites part of the list object's value in place. Because the list object is shared by (referenced from) other variables, though, an in-place change like this doesn't affect only L1—that is, you must be aware that when you make such changes, they can impact other parts of your program. In this example, the effect shows up in L2 as well because it references the same object as L1. Again, we haven't actually changed L2, either, but its value will appear different because it refers to an object that has been overwritten in place. This behavior occurs only for mutable objects that support in-place changes and is usually what you want, but you should be aware of how it works so that it's expected. It's also just the default: if you don't want such behavior, you can request that Python copy objects instead of making references. There are a variety of ways to copy a list, includin

g using the built-in list function, the list copy method, and the standard-library copy module. Perhaps the most common way is to slice from start to finish (see Chapters 4 and 7 for more on slicing):

```
python >>> L1 = [2, 3, 4] >>> L2 = L1[:] # Make a copy of L1 (or list(L1), L1.copy(), etc.) >>> L1[0] = 24 >>> L1 [24, 3, 4]

>>> L2 # L2 is not changed this time: different objects [2, 3, 4]
```

Here, the change made through L1 is not reflected in L2 because L2 references a copy of the object L1 references, not the original; that is, the two variables point to different objects and different pieces of memory. Note that this slicing technique won't work on the other major mutable core types, dictionaries and sets, because they are not sequences—to copy a dictionary or set, instead use their `X.copy()` method call (lists have one too), or pass the original object to their type names, `dict` and `set`. Also, note that the standard-library copy module has a call for copying any object type generically, as well as a call for copying nested object structures—a dictionary with nested lists, for example:

```
python import copy X = copy.copy(Y) # Make top-level "shallow" copy of any object Y X = copy.deepcopy(Y) # Make deep copy of any object Y: copy all nested parts We'll explore lists and dictionaries in more depth
```

h, and revisit the concept of shared references and copies, in Chapters 8 and 9. For now, keep in mind that objects that can be changed in place—that is, mutable objects—are always open to these kinds of effects in any code they pass through. In Python, this includes lists, dictionaries, sets, and some objects defined with class statements. If this is not desired behavior, simply copy your objects as needed.

中文翻译

在本书这一部分接下来的章节中你还会学到更多：有些操作确实会原地（in place）改变对象，但它们只被 Python 的可变（mutable）类型支持——包括列表、字典和集合。例如，对列表某个偏移位置赋值，实际上会原地改变列表对象本身，而不是生成一个全新的列表对象。尽管目前你得在某种程度上先凭直觉相信这一点，但这个区别在你的程序中可能至关重要。对于那些支持原地修改的对象，你需要对共享引用格外警惕，因为从一个名字发出的修改可能会影响到其他名字。否则，你的对象可能“无缘无故”就变了。考虑到所有赋值都基于引用（包括函数实参传递），这是一个无处不在的现象。为了说明，让我们再看一下第 4 章介绍的列表对象。回忆一下：列表支持对某一位置的原地赋值，它就是对其他对象的简单集合，用方括号编写：`python >>> L1 = [2, 3, 4] >>> L2 = L1` 这里的 L1 是一个包含对象 2、3、4 的列表。列表里的元素通过它们的位置偏移来访问，所以 L1[0] 引用的是对象 2，即列表 L1 的第一项。当然，列表本身也是对象，就像整数和字符串一样。执行完上面两条赋值后，L1 和 L2 引用着同一个共享对象，就像前一个

例子中的 a 和 b 一样（见图 6-2）。现在设想，和前面一样，我们把这个交互扩展成下面的语句：`python >>> L1 = 24` 这条赋值只是让 L1 指向一个不同的对象；L2 仍然引用原来的列表，与上一节的情形类似。但是，如果我们稍稍改一下这条语句的语法，它的效果就会发生根本性变化：`python >>> L1 = [2, 3, 4] # 一个可变对象>>> L2 = L1 # 建立对同一个对象的引用>>> L1[0] = 24 # 一次原地修改>>> L1 # L1 变了[24, 3, 4]>>> L2 # 但 L2 也变了! [24, 3, 4]` 实际上，第三步我们并没有改变 L1 本身；我们改变的是 L1 所引用对象的某个组成部分。这种修改会原地覆盖列表对象值的一部分。但由于这个列表对象被其他变量共享（即从它们那里被引用），这样的原地修改影响的不只是 L1——也就是说，你必须意识到：当你做这种修改时，它可能波及程序的其他部分。在这个例子里，效果也出现在了 L2 上，因为 L2 引用着和 L1 同一个对象。再说一遍，我们并没有真的“改变” L2，只是它的值看起来不同了，因为它引用的是那个已被原地覆盖过的对象。这个行为只发生在“支持原地修改的可变对象”身上，而且通常正是你想要的，但你应该了解它的运作方式，以便心中有数。这同时也只是默认行为：如果你不想要这种行为，可以要求 Python 去复制（copy）对象，而不是建立引用。复制列表的方式有很多种，包括使用内置函数 `list`、列表的 `copy` 方法，以及标准库的 `copy` 模块。也许最常用的方法是“从起点切到终点”的切片法（关于切片，详见第 4、7 章）：`python >>> L1 = [2, 3, 4]>>> L2 = L1[:] # 复制一份 L1（也可以用 list(L1)、L1.copy() 等）>>> L1[0] = 24>>> L1 [24, 3, 4]>>> L2 # 这次 L2 没变：是两个不同的对象[2, 3, 4]` 在这里，通过 L1 所做的修改不会反映到 L2，因为 L2 引用的是

"L1 所引用对象"的一份复制品，而不是原件；也就是说，两个变量指向不同的对象、不同的内存。注意：切片这种技法对其他主要可变核心类型（字典和集合）不适用，因为它们不是序列（sequence）。要复制字典或集合，请改用它们的 `X.copy()` 方法（列表也有这个方法），或者把原对象传给类型名 `dict` 和 `set`。另外注意，标准库 `copy` 模块有一个可泛化复制任意对象类型的调用，还有一个可复制嵌套对象结构的调用——比如一个嵌套着列表的字典：`python import copy X = copy.copy(Y) # 复制任意对象 Y 的顶层（"浅拷贝"shallow copy） X = copy.deepcopy(Y) # 深拷贝任意对象 Y：复制所有嵌套部分`我们会在第 8、9 章更深入地探索列表和字典，并重新审视共享引用与复制的概念。眼下请记住：能被原地修改的对象——即可变对象——无论在什么代码里被传递，都始终敞开着这类影响的大门。在 Python 中，这包括列表、字典、集合，以及一些用 `class` 语句定义的对象。如果这不是你想要的行为，按需复制你的对象即可。

深度理解

- 核心概念（本章最重要的意象）：对可变对象，你要区分两种操作——重新绑定（rebinding, `L1 = 24`, 换标签）与原地修改（in-place, `L1[0] = 24`, 改内容）。前者只影响自己；后者只要对象被共享，所有名字都受影响。
- 底层实现：`L1[0] = 24` 执行的是 `STORESUBSCR` 字节码，它取列表对象、按下标定位内部指针槽位、把新对象引用写进去——列表对象本身的地址（`L1` 指向的引用值）没变，变的只是列表内部 "0 号槽" 指向的对象。用"指针

值"表述：L2 手里那把"列表地址"没变，所以 L2 看到的是同一份被改过的名单。

·为什么容易踩坑（实际问题）：函数传参也是引用传递！
`def f(lst): lst.append(x)` 会在函数外"篡改"调用者传入的列表，这是新手最著名的 bug 之一（第 14 章会系统讲）。本书强调"赋值基于引用，包括函数参数传递"——这就是在提前给你埋伏笔。

·对策（复制）：需要独立副本时，选工具要看"深度"：

· `L1[:]`、`list(L1)`、`L1.copy()`：只复制最外层（浅拷贝）。

· `copy.copy(X)`：通用浅拷贝；`copy.deepcopy(X)`：递归复制所有嵌套层（深拷贝）。

·深浅拷贝的具体机制在后续章节展开，这里先建立"共享 vs 复制"的区别即可。

·初学者误区：

· "[:] 为什么能复制？"——切片返回一个新列表对象，元素是原元素的引用，所以叫浅拷贝。若列表里是可变元素（如子列表），改动子元素仍会"穿透"。

· "字典/集合为什么不能用切片复制？"——它们不是序列，没有 `[::]` 语法，请用 `.copy()` 或 `dict()/set()`。

·以为"浅拷贝后两层就安全了"——`copy.deepcopy` 才保证完全隔离。

·生活例子：一张全班的点名单（列表对象）被你和班长各抄了一份"名单引用"（同一张纸）。你在纸上把张三改成李四（原地修改）——班主任手里的"同一张纸"也变了。如果你复印了一份背面的副本（复制），你在副本上改，

原件不受影响；但这副本若是浅拷贝，复印时"每个人的详档"仍是原件，你改详档依然会"连坐"。

6.8 共享引用与相等性 (Shared References and Equality)

英文原文

In the interest of full disclosure, it's worth pointing out that the garbage-collection behavior described earlier in this chapter may be more conceptual than literal for certain types. Consider these statements:

```
python >>> x = 99 >>> x = 'Python' # Reclaim 99 now?
Because Python caches and reuses small integers and small strings, as mentioned earlier, the object 99 here is probably not literally reclaimed; instead, it will likely remain in a system table to be reused the next time you generate a 99 in your code. Most kinds of objects, though, are reclaimed immediately when they are no longer referenced; for those that are not, the caching mechanism is irrelevant to your code—unless you use atypical tools. For instance, because of Python's reference model, there are two different ways to check for equality in a Python program. Let's create a shared reference to demonstrate:
```

```
python >>> L = [1, 2, 3] >>> M = L # Make M and L reference the same object
>>> L == M # Same value
```

s True

```
>>> L is M # Same objects True
```

The first technique here, the `==` operator, tests whether the two referenced objects have the same values; this is the method almost always used for equality checks in Python. The second method, the `is` operator, instead tests for object identity—it returns `True` only if both names point to the exact same object, so it is a much stronger form of equality testing and is rarely applied in most programs (except for single-instance objects like `None`, `True`, and `False`, as in the prior chapter). Really, `is` simply compares the pointers that implement references, and it serves as a way to detect shared references in your code if needed. It returns `False` if the names point to equivalent but different objects, as is the case when we run two different literal expressions:

```
python >>> L = [1, 2, 3] >>> M = [1, 2, 3] # Make M
and L reference different objects >>> L == M # Same
values True
```

```
>>> L is M # Different objects False
```

But now watch what happens when we perform the same operations on an immutable object like an integer:

```
python >>> X = 99 >>> Y = 99 # Should be two different
objects >>> X == Y True
```



```
>>> X is Y # Same object anyhow: caching at work! True
```

In this interaction, `X` and `Y` should be `==` (same value), but not `is` (same object) because we ran two different literal expressions (`99`). Because some integers and strings are cached and reused, though, it tells us they reference the same single object. If you really want to look under the hood, the `id` built-in is another way to check object identities (and may or may not return an object's address in memory); and the `getrefcount` function in the standard `sys` module returns the passed object's reference count. As of Python 3.12, however, the latter is not as interesting as it once was, because it returns a very high count for objects considered to be immortal—which in Python just means cached for reuse:

```
python >>> import sys >>> sys.getrefcount(99) # 99 is immortal (cached) 4294967295
```

```
>>> sys.getrefcount(21000) # But this is not 1
```

```
>>> id(99) == id(99) # Same ID/same object (address?) True
```

This object caching and reuse is irrelevant to your code (unless you run the `is` check!). Because you cannot change immutable numbers or strings in place, it doesn't matter how many references there are to the same object—every reference will always see the same, unchanging value even if they all reference the same

cached object. Still, this behavior reflects one of the many ways Python optimizes its model for execution speed.

中文翻译

本着"把一切交代清楚"的原则，值得指出：本章前面描述的垃圾回收行为，对某些类型来说可能"概念性多于字面性"。看这些语句：`python >>> x = 99 >>> x = 'Python' # 现在回收 99?`如前所述，因为 Python 会缓存并复用小整数和小字符串，这里的对象 99 很可能并没有被字面上回收；相反，它很可能会留在系统的一张表里，等你在代码中下次再产生一个 99 时被复用。不过，大多数对象在不再被引用时确实会被立即回收；对于那些不被回收的，这个缓存机制对你的代码无关紧要——除非你使用了非常规的工具。例如，正因为 Python 的引用模型，在 Python 程序中有两种不同的方式来判断相等性。让我们建立一个共享引用来演示：`python >>> L = [1, 2, 3] >>> M = L # 让 M 和 L 引用同一个对象 >>> L == M # 值相同 True >>> L is M # 对象相同 True`第一种方式 `==` 运算符，测试两个被引用对象是否具有相同的值 (value)；这是 Python 中几乎总是使用的相等性检查方法。第二种方法 `is` 运算符，则测试对象的身份 (identity) ——只有当两个名字指向完全相同的那个对象时它才返回 True。所以它是一种强得多的相等性测试，在大多数程序中很少被使用（除了像 None、True、False 这类单实例对象，如前一章所述）。实际上，`is` 只比较了一个东西——实现引用的那些指针，如果需要在代码中检测共享引用，它就是一种手段。当两个名字指向"值等价但对象不同"的对象时，`is` 返回 False——当我们运行两

条不同的字面量表达式时就是这样：python >>> L = [1, 2, 3] >>> M = [1, 2, 3] # 让 M 和 L 引用不同的对象 >>> L == M # 值相同 True >>> L is M # 但对象不同 False 但是，现在来看看我们对整数这样的不可变对象做同样操作时会发生什么：python >>> X = 99 >>> Y = 99 # 按理说是两个不同的对象 >>> X == Y True >>> X is Y # 可它们居然是同一个对象：缓存机制生效了！True 在这段交互中，X 和 Y 本应该 ==（值相同）但不 is（对象不同），因为我们运行了两条不同的字面量表达式（都是 99）。然而由于一些整数和字符串会被缓存并复用，is 告诉我们它们引用的是同一个对象。如果你真想钻到引擎盖下面看，内置函数 id 是检查对象身份的另一个途径（它可能返回、也可能不返回对象的内存地址）；标准库 sys 模块里的 getrefcount 函数会返回传入对象的引用计数。不过，从 Python 3.12 开始，后者的趣味性大不如前，因为对于被认为是“不朽”（immortal）的对象——在 Python 中就是“被缓存待复用”的意思——它返回的计数非常高：python >>> import sys >>> sys.getrefcount(99) # 99 是不朽的（被缓存）4294967295 >>> sys.getrefcount(21000) # 但这个不是 1 >>> id(99) == id(99) # 相同的 ID / 相同的对象（地址？）True 这种对象缓存与复用对你的代码无关紧要（除非你运行 is 检查！）。因为你无法原地修改不可变的数字或字符串，所以不论同一个对象有多少个引用都无所谓——即便所有引用都指向同一个缓存对象，每个引用看到的也永远是同一个不变的值。不过，这个行为恰恰反映了 Python 为执行速度优化其模型的众多方式之一。

深度理解

·核心概念（全书最重要的一对）：`==` 比值（value），`is` 比身份/地址（id）。`is` 本质是比较两个引用（指针）是否指向同一内存。

·对象三要素全部到齐：`==` 走 value；`is` 走 id；type 决定你能做什么。判断两个对象“一样吗”，永远先问自己“你要的是值相同还是就是同一份”。

·底层实现：`is` 在 CPython 里就是 `p1 == p2` 的指针比较，几乎零开销；`==` 则会调用对象的特殊方法 `eq`，可能做递归深比较（列表较长时开销明显）。`getrefcount(99)` 返回的 4294967295 是 `UINTMAX` 这样的“哨兵值”，表示：这个对象被标记为 `immortal`（不朽/永久对象），解释器不再试图管理它的生命周期——这是 CPython 3.12 引入的优化。

·设计原因/为什么值得记牢：

1. 用 `is` 判断“是同一个共享对象”可以用来做触发式缓存清理、对象数量审计；（补充）正规写法是 `is None`、`is True`，因为 `None/True/False` 是保证唯一的单例，`is` 既快又安全。

2. 小整数与短字符串的缓存（intern/caching）属于解释器实现细节——不该作为你的程序逻辑依赖（跨实现不同，比如 Jython 就不是这样）。

·实际问题：缓存让“99 是同一个对象”成立，但 `-5 ~ 256` 之外的大整数通常不被缓存，`300 is 300` 可能（每次新建字面量时）是 `False`——所以永远不要用 `is` 比较数字/字

符串值，只用于 None/True/False。这是面试与代码评审的经典考点。

·初学者误区：

·误用 is 比较内容 → 结果诡异（如用 is 比较两个“值相同”的字符串，恰好碰运气）；正确做法：内容一律 ==，单例用 is。

·误以为“== 一定 is”——L == M 为 True 的两个独立列表，is 是 False（书本原例）。

·误以为引用计数一定是实时可见的——缓存对象/不朽对象会给出“假”计数，不要拿它做程序逻辑。

代码分析

```
>>> L = [1, 2, 3]
>>> M = L           # M L
>>> L == M          #
True
>>> L is M          #
True
```

·解释器如何处理：M = L 把 L 指向的列表地址复制给 M，此时两个名字存的是同一个地址。L == M：调用 list 的 eq，比较两者内容逐项相等 → True；L is M：直接比较两个地址是否相等 → 同一地址 → True。

```
>>> X = 99
>>> Y = 99          #
>>> X == Y          # True
>>> X is Y          # True
```

·解释器如何处理：每个 99 字面量先查“小整数缓存”（CPython 对 -5~256 预分配并缓存），命中就直接复用同一

个对象——所以两个字面量拿到同一个地址，is 为 True；若 X = 300; Y = 300，多数 CPython 中不缓存 300，两条字面量各自新建对象，is 为 False。这解释了为什么不把 is 用于值比较。

```
>>> import sys
>>> sys.getrefcount(99)          # 99
4294967295
>>> sys.getrefcount(2 ** 1000) #
1
```

解释器如何处理：Python 3.12 起，被缓存/复用概率极高的对象被标记为"不朽"，getrefcount 对它们直接返回哨兵值 UINTMAX (4294967295) 而不是真实计数；对普通对象（如 21000）则返回真实引用计数——临时对象只在本次表达式中被引用 1 次，所以是 1。注意：图中 id(99) == id(99) 为 True，说明两次取 id 是同一个地址——但这也恰恰是"缓存导致 is 为真"的证据链。

6.9 动态类型无处不在 (Dynamic Typing Is Everywhere)

英文原文

Of course, you don't really need to draw name/object diagrams with circles and arrows to use Python. When you're starting out, though, it sometimes helps you understand unusual cases if you can trace their reference structures as we've done here. If a mutable object changes out from under you when passed around y

our program, for example, chances are you are witnessing some of this chapter's subject matter firsthand. Moreover, even if dynamic typing seems a little abstract at this point, you probably will care about it eventually. Because everything seems to work by assignment and references in Python, a basic understanding of this model is useful in many different contexts. As you'll see, it works the same in assignment statements, function arguments, for loop variables, module imports, class attributes, and more. The good news is that there is just one assignment model in Python; once you get a handle on dynamic typing, you'll find that it works the same everywhere in the language. At the most practical level, dynamic typing means there is less code for you to write. Just as importantly, though, dynamic typing is also the root of Python's polymorphism, a concept introduced in Chapter 4 that we'll revisit again later in this book. Because we do not constrain types in Python code, it is both concise and highly flexible. As you'll see, when used well, dynamic typing—and the polymorphism it implies—produces code that automatically adapts to new requirements as your systems evolve.

中文翻译

当然，你并不真的需要画带圆圈和箭头的名字/对象图才能使用 Python。不过，当你刚开始学习时，如果能像我们这样追踪引用结构，有时会有助于你理解不寻常的情况。比如

，当一个可变对象在程序中被传来传去，在你脚底下悄悄改变了，那么很可能你正亲眼目睹本章内容的活生生证据。此外，即使此刻动态类型看起来还有些抽象，你最终很可能会有意无意在意它。因为在 Python 中，一切似乎都是通过赋值和引用来工作的，对这一模型的基本理解在许多不同场景中都很实用。正如你将看到的，它在赋值语句、函数参数、for 循环变量、模块导入、类属性等地方都以同样的方式工作。好消息是：Python 中只有一个赋值模型；一旦掌握了动态类型，你会发现它在语言的任何地方都同样地工作。在最实际的层面，动态类型意味着你要写的代码更少。但同样重要的是，动态类型也是 Python 多态的根源——这个概念在第 4 章引入，本书后面还会再次探讨。因为我们在 Python 代码中不约束类型，代码既简洁又高度灵活。正如你将看到的，用得好的话，动态类型——及其隐含的多态——会产出随系统演进自动适应新需求的代码。

深度理解

- 核心概念：动态类型不是某个角落里的小机制，而是覆盖全语言的统一赋值模型。列表、函数、循环、导入、类属性.....都是"名字→对象"的同一套把戏。
- 底层实现：字节码层面，全局/局部变量经 STORENAME /LOADNAME (或 STOREFAST/LOADFAST)、函数参数经 STOREFAST、类属性经 STOREATTR、模块导入经 IMPORTNAME 后同样 STORE——全部走同一条"建立名字引用"逻辑。这就是"一个模型，处处适用"的机器证据。
- 设计原因（多态的正反馈）：不约束类型 → 函数天然接受任何对象 → 只要对象支持所需操作就能用（鸭子类型

萌芽)。系统加需求时，往往是新增一种切合既有接口的对象类型塞进去，老代码零改动。

·实际问题：调试"对象怎么在我眼皮底下变了"时，按"引用结构追踪法"（画出谁指向谁）比瞎猜高效得多——这是作者反复训练读者的基本功，也是后来调试递归、闭包、回调的大杀器。

·初学者误区：① 以为"循环变量在每次迭代会被复制一份"——for x in lst 的 x 每次只是被重绑到 lst 当前元素对象（同样的引用模型）；② 以为"函数参数是复制传入"——其实是共享引用，改可变参数就会波及外部；③ 以为"import 是复制模块"——导入绑定的是模块对象这个引用的一个名字，不同模块里 import 同一模块得到同一个对象。

·生活例子："只有一个模型"就像你学了一种交通工具的操作后，发现汽车、地铁、自行车都得会"转向"这个同一动作——踏板换脚踏板，但技能通用。动态类型则是"送货员不认识包裹内容也能送"——收货人自己检查里面是什么。

6.10 类型标注：可选、不使用、为什么？ (Type Hinting: Optional, Unused, and Why?)

英文原文

Finally, an implausible plot twist. If you've read Python code written in recent years, you may have stumble

d across some type declarations for variable names that look like the following—and seem curious and out of place for a dynamically typed language like Python, and at first glance contradictory to some of this chapter's claims: `python >>> a: int >>> b: int = 0 >>> c: list[int] = [1, 2, 3]` As previewed in Chapter 4, this is known as type hinting. Syntactically, it takes the form of a colon and object type, between a variable and an optional assignment. The object type can be a name or an expression to denote collections (`list[int]` means a list of integers) and can use names predefined in the standard-library typing module (e.g., `Iterable`, `Union`, and `Any`) to express richer types per elaborate theory. As of Python 3.12, a new type statement can even define type aliases to use in hints, though simple assignments that predated it can too: `python >>> type Data = list[float] >>> Data = list[float]` As also noted in Chapter 4, though, type hints are optional, unused, and largely academic. Python does not require them and does not use them in any way and has no intentions of ever doing so. They are meant solely for use in third-party tools like type checkers, and as a form of documentation that's an alternative to code comments. You can say the same things more simply in both `#` comments and documentation strings you'll meet later. Even when used, type hints do not constrain your code's types in any way. The preceding type hint for `a`, for instance, does not create name `a` (only

assignment does), and b's and c's hints are not enforced in the least: python >>> a NameError: name'a' is not defined >>> b='hack'>>> c='code'>>> b, c ('hack','code') Type hints can also appear in definitions of functions (and class methods) to document types of parameters and results, commandeering an earlier feature known as function annotations. We haven't covered these yet, but as a preview, the following function hints that it accepts an integer and list of strings and returns a float—extraneous info that shows up in annotations dictionaries of hosting objects: python >>> def func(a: int, b: list[str]) -> float: ... return'anything' + a + b Yet as for simple variables, these hints are fully unused, and anything goes when this function is actually run. Strings, for example, work fine for both inputs and outputs, despite the seemingly rigid hints: python >>> func('You','Want')'anythingYouWant' That is, type hinting is a conceptually heavy tool adopted by Python but completely unused by Python. It's at best just another form of documentation in Python itself, albeit one that comes with complex rules. External tools might use type hints to check for type mismatches (e.g., mypy) or boost performance, but such tools are also optional, uncommon, and wholly separate from the Python language. Furthermore, programs require runtime testing in any language, and optimized Pythons introduced in Chapter 2 do not use type hints today, and in some cases cannot (see PyPy

). More to the point, though, type hinting is also completely at odds with Python's core notion of dynamic typing. Type declarations in a dynamically typed language are a pointless paradox that negates much of Python's value proposition. Teaching this bizarre extension to Python learners would be a disservice to both Python and learners. Hence, this book recommends that beginners avoid type hinting at least until they are comfortable with Python's dynamic-typing paradigm. This book also won't be covering it further, because it's far too much extra heft sans benefit for newcomers struggling to master Python's already sizable fundamentals. If and when you opt to delve into this inane yet convoluted corner of Python, consult its docs for more information. In the end—and despite what you may see in Python code written by programmers coming from other languages—type hinting does not mean that Python is statically typed. Python still uses only dynamic typing, and hopefully always will. After all, this is the root of most of its advantages over other tools. Let's hope that Python developers of the future learn this well before bloating or breaking a tool used and beloved by millions.

中文翻译

最后，一个难以想象的剧情反转。如果你读过近几年写的 Python 代码，你可能会撞见下面这种对变量名的类型声明——看起来对于一个像 Python 这样动态类型的语言既新

奇又格格不入，乍一看还与本章的一些主张互相矛盾：python >>> a: int >>> b: int = 0 >>> c: list[int] = [1, 2, 3] 如第 4 章预告的那样，这被称为类型标注（type hinting）。在语法上，它采取“冒号 + 对象类型”的形式，位于一个变量和一个可选的赋值之间。对象类型可以是一个名字，也可以是一个表示集合的表达式（list[int] 意思是“整数的列表”），还可以使用标准库 typing 模块中预定义的名字（如 Iterable、Union、Any），以便按繁复的理论表达更丰富的类型。从 Python 3.12 起，一个新的 type 语句甚至能定义供标注使用的类型别名，虽然早于它的普通赋值也能做到：python >>> type Data = list[float] >>> Data = list[float] 但正如第 4 章也指出的，类型标注是可选的、不被使用的、大体上是纯学术性的。Python 不要求它们，不以任何方式使用它们，也从未打算这样做。它们唯一的用途，是在诸如类型检查器（type checker）之类的第三方工具里，以及作为注释之外的一种文档形式。相同的内容，你可以更简单地写在 # 注释里，或者写在你以后会遇到的文档字符串（docstring）里。即便使用了，类型标注也绝不以任何方式约束你代码中的类型。例如，前面给 a 的类型标注并不会创建名字 a（只有赋值才会创建），而 b 和 c 的标注也完全不被强制执行：python >>> a NameError: name'a' is not defined >>> b = 'hack' >>> c = 'code' >>> b, c ('hack','code') 类型标注还可以出现在函数（以及类的方法）的定义里，用来记录参数和结果的类型，这“征用”了早期一个名为函数注解（function annotations）的特性。我们还没讲到这里，但先预览一下：下面这个函数标注自己“接收一个整数和一个字符串列表，返回一个浮点数”——这是多余的信息，它出现在宿主对象的 annotations

字典中：`python >>> def func(a: int, b: list[str]) -> float: ... return 'anything' + a + b` 然而与普通变量一样，这些标注完全不被使用，这个函数真正运行时什么都行。例如，字符串作为输入和输出都工作得很好，尽管那些标注看起来似乎很严格：`python >>> func('You','Want')anythingYouWant` 也就是说，类型标注是一个概念上很重的工具——被 Python 采用，却被 Python 完全不使用。在 Python 自身看来，它至多只是另一种形式的文档，只不过附带复杂的规则。外部工具或许会用类型标注来检查类型不匹配（如 mypy）或提升性能，但这类工具同样是可选的、不常见的，且与 Python 语言完全分离。此外，任何语言都需要运行时测试，第 2 章介绍的那些“优化版 Python”目前都不使用类型标注，有些甚至不能使用（参见 PyPy）。更要紧的是，类型标注与 Python 动态类型的核心观念完全相悖。在一个动态类型语言里做类型声明，是一个毫无意义的悖论，否定了 Python 的大部分价值主张。向 Python 学习者传授这种古怪的扩展，无论对 Python 还是对学习者的都是伤害。因此，本书建议初学者至少要在对动态类型范式感到舒适之前，避免使用类型标注。本书也不会再进一步介绍它，因为对还在努力攻克本就庞大的基础知识的初学者来说，它带来的额外负担与实际收益完全不成比例，实在太多了。如果哪天你自愿钻进 Python 这个“既荒诞又绕”的角落，请查阅它的文档获取更多信息。归根结底——不管你可能在那些由其他语言转来的程序员所写的 Python 代码里看到什么——类型标注意味着 Python 是静态类型的吗？不。Python 仍然只使用动态类型，希望永远如此。毕竟，这正是它相对于其他工具的大部分优势的根源。愿未来的 Python 开发者们能深刻理解这一点，在把这个被数百万人使用和喜

爱的工具弄臃肿或弄坏之前。

深度理解

- 核心概念：类型标注 = 文档（给人和工具看），不是约束（给解释器用）。它不创建名字、不强制类型、不影响运行行为。
- 底层实现：`a: int` 只在模块/函数命名空间中往 `annotations` 字典里写一条 `{'a': int}` 的元数据，然后.....就完了。运行时没有任何类型检查字节码被插入。`type Data = list[float]` 是 3.12 的新语法糖，同样只在一处登记别名。
- 设计原因（本书立场）：Lutz 的观点旗帜鲜明——动态类型是 Python 的"价值主张"和"优势之源"，在动态类型语言里引入"类型宣言"是一个自相矛盾的悖论。他并非否定静态类型（C/C++/Java 有价值），而是认为类型标注与 Python 的存在理由相冲突，且对初学者来说"包袱大于收益"。注意这是 Lutz 的个人观点——社区里很多开发者支持注解（配合 `mypy` 做静态检查），你在真实项目里仍常见到 `Ctrl` 自动补全依赖它工作。
- 实际问题（分歧要诚实讲）：
 - Lutz 的立场 vs 业界现状：大型项目（如大型 Django/FastAPI 项目）常强制使用 `mypy`；编辑器提示、库的智能补全也常从标注获益。所以"标注没用"是针对 Python 语言的运行时而言是事实，但对"工程生态"而言它很有用。
 - 如果你将来要进团队协作项目，大概率要用上；本书仅为初学者阶段做出"先别学"的建议。两者不矛盾：先打牢动态类型的底子，再学标注就不会被误导。

·初学者误区：

·以为 `b: int = 0` 是"声明变量的类型"——`b = 'hack'` 完全合法，改判无需任何许可。

·以为标注能抓 bug——运行时它抓不了；静态检查靠外部工具（`mypy`、`pyright`）。

·以为 `a: int` 会创建 `a`——不会，`NameError` 依旧（书中原例）。

·生活例子：类型标注像超市购物车上的"标签卡"（"这格放苹果"）。收银员（Python 运行时）根本不看标签，你把橘子放进"苹果格"它照样扫码；只有质检员（`mypy`，第三方工具）路过时才会皱眉。

代码分析

```
>>> a: int                                     #
>>> a
NameError: name 'a' is not defined
```

·解释器如何处理：`a: int` 被解析成"带注解的名字"，在执行时把 `int` 类型对象登记进 `annotations`，但不会向符号表 `STORENAME`——所以随后访问 `a` 仍然 `NameError`。这干净地证明了"只有赋值才创建名字"。

```
>>> def func(a: int, b: list[str]) -> float:
...     return 'anything' + a + b
>>> func('You', 'Want')
'anythingYouWant'
```

·解释器如何处理：函数定义时把参数/返回值标注放进 `func.annotations`（如 `{'a': int, 'b': list[str], 'return': float}`），之后调用函数时没有任何校验步骤：`a='You'`、`b='W`

ant' 都是以二进制字符串直接参与 +, 返回'anythingYouWant'——标注既被忽略, 又挡不住任何类型。这直观演示了"概念重、运行时零影响"。

```
>>> type Data = list[float]    # Python 3.12
>>> Data = list[float]         #
```

解释器如何处理: 3.12 的 type 语句把等号右边也登记为 annotations 里的类型别名; 而第二行普通赋值 Data = list[float] 只是让名字 Data 引用类型对象 list[float], 两者运行时效果一致——再次印证"Python 只是给名字挂对象, 类型标注不参与运行"。

6.11 本章小结 (Chapter Summary)

英文原文

This chapter took a deeper look at Python's dynamic typing model—that is, the way that Python keeps track of object types for us automatically, rather than requiring us to code declaration statements in our scripts. Along the way, we learned how variables and objects are associated by references in Python that enable type flexibility. We also explored the topic of garbage collection, learned how shared references to mutable objects can affect multiple variables, and saw how references impact the notion of equality in Python. Lastly, we briefly glimpsed type hinting—a subdomain that weirdly adds unused type declarations to a dynamically typed language. Because there is just one a

assignment model in Python, and because assignment pops up everywhere in the language, it's important that you have a handle on the model before moving on. The following quiz should help you review some of this chapter's ideas. After that, we'll resume our core object tour in the next chapter, with strings.

中文翻译

本章深入考察了 Python 的动态类型模型——也就是说，Python 如何替我们自动跟踪对象的类型，而不是要求我们在脚本里编写声明语句。一路走下来，我们学到了 Python 中变量与对象如何通过引用关联，正是这种关联成就了类型灵活性。我们还探讨了垃圾回收这个话题，学习了对可变对象的共享引用如何影响多个变量，并看到引用如何影响 Python 中"相等性"的概念。最后，我们匆匆瞥了一眼类型标注——一个奇怪地在动态类型语言上叠加"不被使用的类型声明"的领域。因为 Python 中只有一个赋值模型，也因为赋值在语言中无处不在，所以在继续之前，你必须掌握这个模型。下面的测验能帮你复习本章的一些观点。之后，我们将在下一章回归核心对象之旅，对象是字符串。

深度理解

·核心概念：第 6 章是整本书的"理论地基"——引用模型 = 变量（名字）恒为"指向对象的引用"，对象自带 value/type/id，回收靠引用计数，共享靠多个名字指同一地址，相等有"值"与"身份"两条评判线。下一阶段（第 7 章起）每一类对象你都会拿这套模型去套。

·为后续铺路：值得提前告诉你几个"复用同一个模型"的经典场景（均会在书中对应章节展开）：函数参数按引用传（第 14 章）、可变默认参数陷阱（第 14 章）、闭包与自由变量引用（第 17 章）、类属性与实例属性、模块的单例导入。先掌握本模型，会让这些后面自然贯通。

·生活例子：这一章的内容像"认识你真的在用电脑之前，先认识这台电脑的接线图"——接线规则就几条，但全机只此一套。

6.12 自我测验 (Test Your Knowledge: Quiz)

英文原文

1. Consider the following three statements. Do they change the value printed for A?

```
python A = 'code' B = A
```

```
B = 'Python'
```

2. Consider these three statements. Do they change the printed value of A?

```
python A = ['code'] B = A B[0] = 'Python'
```

3. How about these—is A changed now?

```
python A = ['code'] B = A[:] B[0] = 'Python'
```

中文翻译

1. 考虑下面三条语句。它们会改变打印出来的 A 的值吗？
python A = 'code' B = A B = 'Python'
2. 考虑这三条语句。它们会改变打印出来的 A 的值吗？
python A = ['code'] B = A B[0] = 'Python'
3. 下面这几条呢——这次 A 变了吗？
python A = ['code'] B = A[:] B[0] = 'Python'

深度理解（解题心法）

· 三道题的共同玩法：先问“A 和 B 是否共享对象”，再问“之后动的是谁”。

· 题 1：B = A 共享字符串'code'；B = 'Python' 是重绑 B（换标签），只影响 B，A 仍是'code'。答案：不变。

· 题 2：B = A 共享同一个列表；B[0] = 'Python' 是原地修改共享对象内容，A 看到同样的变化。答案：变为 ['Python']。

· 题 3：B = A[:] 是复制一份新列表；B[0] = 'Python' 只改 B 那份，A 是原件不受影响。答案：仍为 ['code']。

· 考点归纳：字符串是不可变对象，“赋值 = 换新对象”永远安全；列表是可变对象，区分“重绑名字”与“原地改内容”决定共享是否穿透。

6.13 自我测验：答案（Test Your Knowledge : Answers）

英文原文

1. No: A still prints as 'code'. When B is assigned to the string 'Python', all that happens is that the variable B is reset to point to the new string object. A and B initially share (i.e., reference/point to) the same single string object 'code', but two names are never linked together in Python. Thus, setting B to a different object has no effect on A. The same would be true if the last statement here were `B = B + 'coding'`, by the way—the concatenation would make a new object for its result, which would then be assigned to B only. We can never overwrite a string (or number, or tuple) in place, because strings are immutable.

2. Yes: A now prints as ['Python']. Technically, we haven't really changed either A or B; instead, we've changed part of the object they both reference (point to) by overwriting that object in place through the variable B. Because A references the same object as B, the update is reflected in A, too.

3. No: A still prints as ['code']. The in-place assignment through B has no effect this time because the slice expression made a copy of the list object before it was assigned to B. After the second assignment statement, there are two different list objects that have the same value—in Python, we say they are `=`, but not `is`. The third statement changes the value of the list object pointed to by B, but not that pointed to by A.

中文翻译

1. 不会：A 仍然打印为 'code'。当 B 被赋为字符串 'Python' 时，所发生的一切只是变量 B 被重置为指向新的字符串对象。A 和 B 最初共享同一个字符串对象 'code'，但在 Python 中两个名字永远不会被“链接”在一起。因此，把 B 设为另一个对象对 A 毫无影响。顺带一提，如果最后一条语句是 `B = B + 'coding'`，结果也一样——拼接会为结果创建一个新对象，然后只赋值给 B。我们永远无法原地覆盖一个字符串（或数字、或元组），因为字符串是不可变的。

2. 会：A 现在打印为 ['Python']。严格来说，我们并没有真正改变 A 或 B 中的任何一个；我们是通过变量 B 完成了“原地覆盖那个对象”——改的是它们俩共同引用的那个对象的一部分。由于 A 引用与 B 相同的对象，这个更新也反映在了 A 上。

3. 不会：A 仍然打印为 ['code']。这次通过 B 的原地赋值没有效果，因为切片表达式在把对象赋给 B 之前，已经先制作了这个列表对象的副本。第二条赋值语句之后，存在两个值相同但不同的列表对象——在 Python 中我们说它们是 `==`，但不是 `is`。第三条语句改变了 B 所指列表对象的值，但没有改变 A 所指的那一个。

深度理解

· 三题答案合起来正好勾勒了本章全貌：不可变（题 1）→ 重绑安全；可变 + 共享（题 2）→ 原地修改穿透；可变 + 复制（题 3）→ 切片制造隔离。

· 题 1 的补充是精华：即便写成 `B = B + 'coding'`，字符串拼接也创造新对象，绝无可能原地改旧字符串——因为 `str`（以及数字、元组）的 `setitem` 之类修改方法根本不存

在。这解释了为什么"不可变对象不怕共享"。

·题 3 直接引出"值相等但身份不同" (== 不是 is) 的概念——为第 6.8 节做了最自然的收尾验证。

6.14 技术侧栏：当引用"变弱"时 (WHEN REFERENCES ARE "WEAK")

英文原文

You may occasionally see the term "weak reference" in the Python world. No, this term isn't a judgment about inferiority. It refers to a somewhat obscure and advanced tool, which is related to the reference model we've explored here, and like the `is` operator, can't really be understood without it. In short, a weak reference, implemented by the `weakref` standard-library module, is a reference to an object that does not by itself prevent the referenced object from being garbage-collected. If the last remaining references to an object are all weak references, the object can be reclaimed. When this happens, the weak references to it will be notified that the object no longer exists and can respond as needed. As an example of its utility, this can be useful in nonessential caches of large objects primarily used elsewhere. If such a cache uses normal references, the cache's references alone would keep the objects in memory indefinitely. By using weak references, the object's space may be reclaimed when it's no

longer needed for its primary role, and the cache will be notified of its demise, either on next fetch or by callback. Not all object types can be weakly referenced, though support can be added for some with OOP techniques we won't explore till later in this book. Still, this is really just a special-case extension to the reference model we met here. For more details on weak references, see Python's library-manual coverage of `weakref`, a useful—if unhappily named—tool.

中文翻译

你偶尔会在 Python 世界里看到“弱引用”（weak reference）这个术语。别误会，这个词不是在贬低“它很差”。它指的是一种有点冷门、高级的工具，与我们在这里探索的引用模型相关，而且和 `is` 运算符一样，不先理解前面这些内容就根本没法理解它。简而言之，弱引用由标准库 `weakref` 模块实现，是一种“本身并不会阻止被引用对象被垃圾回收”的引用。如果一个对象剩下的引用全部都是弱引用，那么这个对象就可以被回收。当这种情况发生时，指向它的那些弱引用会被通知“对象已经不存在了”，并按需做出响应。举一个它的用武之地的例子：这在“主要为别处所用的大对象的非必要缓存”中会很有用。如果这样的缓存用普通引用，光是缓存本身的引用就会让这些对象永远待在内存里。改用弱引用后，当对象不再需要扮演其主要角色时，它的空间可以被回收，而缓存会得知它的消失——要么在下次获取时，要么通过回调函数。并非所有对象类型都能被弱引用，不过借助本书后面才会探索的面向对象（OOP）技术，其中一些类型可以加上这种支持。说到底，这只是我们在本章所遇

引用模型的一个特例扩展。更多关于弱引用的细节，参见 Python 库手册中有关 weakref 的章节——一个很有用、可惜名字起得不怎么样的工具。

深度理解

- 核心概念：普通引用"保命"（`refcnt + 1`，对象活着）；弱引用"围观但不保命"——它记录对象地址，但不计入引用计数。对象回收时，弱引用的"回调/查询"会发现目标已死。
- 底层实现：`weakref.ref(obj)` 内部持有指向对象的指针，但通过专门的 mechanism 不对 `obrefcnt` 加 1；对象析构时，解释器会"回调"所有指向它的弱引用对象，将其置空（返回 `None`）或触发回调。因为要挂在对象头部（`weak list` 槽位），所以列表等内置类型默认不可弱引用（经典例外：`list` 不能，但 `curl` 我能能，这要看文档），而自定义类的实例常可以。
- 设计原因：解决"缓存把大对象留死"的经典问题。纯普通引用缓存 = 对象永远不会回收；弱引用缓存 = 对象主用方挪走了，缓存自动失效，不让缓存成为内存的"终身保外险"。
- 实际问题：典型应用：LRU 缓存、观察者模式的事件订阅、数据库连接池的"关闭即失效"回呼、循环引用场景下避免互保。生产中用 `weakref.WeakValueDictionary` 做与生命周期无关的内存友好索引很常见。
- 初学者误区：① 把"弱引用"理解为"引用计数减一"——不，它只是"不增加计数"，两者体验相似但机制不同；②

以为任何类型都可弱引用——list、dict（默认）不行，tuple 不行；③ 以为它像 del 一样"主动删"——它不删，只是"不拦着删除"。

·生活例子：普通引用是你"手抓着的绳"，人在绳子断以前你不会掉下去；弱引用像"备注里写着'遇到 3 号便还给它'"——你不拽着它，货主（回收机制）想拿走就拿走，然后你再看备注就知道"3 号没了"。

本章总结

这一章没有介绍任何新对象，却教给了你理解所有对象的那把钥匙。读完本章，你应该能闭眼画出 $a = 3$ 、 $b = a$ 、 $L[0] = 24$ 在内存里的样子了。

技术拓展 (Technical Expansion)

实际项目中的应用场景

· aliasing（别名）排查：大型项目里"一个对象不知何时被改了"的 bug，九成是共享引用。排查手法就是本章教的"画引用结构图"：找出对象被哪些名字/容器/闭包引用，逐个掐断或用 copy 建立隔离副本。

·防御性拷贝：对外暴露可变内部状态（如返回内部 list/dict）前先 deepcopy，避免调用方意外"污染"自身数据——这是 API 设计高手的习惯。

·不可变对象大量共享：str、tuple、frozenset 因不可变可安全共享，大量使用它们能显著节约内存并减少副本——

—配合"对象三要素"你能判断何时共享、何时复制。

· is 的合法用途：判 None/True/False、缓存/枚举/枚对象单例比较、sys.intern 后的字符串比较（高级）。别拿它做数值比较。

·弱引用做缓存：weakref.WeakValueDictionary 做"过期自动消失的索引"，避免缓存拖垮内存；gc 模块用于定位循环引用与内存泄漏。

与其他语言的区别

维度	Python（动态类型）	Java / C++（静态类型）
变量是否有类型	无类型，只是"引用"	有固定类型（Java 强类型声明）
类型信息存放	对象头部（type descriptor）	编译期类型系统 + 运行时类型表（RTTI）
何时做类型检查	运行时，操作发生那一刻	编译期多数检查 + 运行时少量（cast）
内存管理	引用计数 + 循环检测 GC	Java：GC（分代）； C++：手动 new/delete（RAII 例外）
相等性	==（值）vs is（身份）	equals() vs ==（引用）——思路惊人相似
声明需求	a = 3 即用	需 int a = 3; 提前声明
可以与"指针"类比	自动解引用的"void"	C++ 才有真指针，需手动解引用与释放

类比 JPython/PyPy 等替代实现：引用计数并非语言规范要求，只有 CPython 严格执行；PyPy 用的是 GC 策略，行为大体一致但不保证即时。

Python 发展历史背景

·动态类型的哲学根基在 Python 创立之初就不变：Guido

van Rossum 是在 ABC 语言的基础上设计的，ABC 就以 "无声明、自动内存管理" 闻名；Python 继承了这条血脉，并在 PEP 20 (The Zen of Python) 中凝结为 "显式优于隐式" "简单优于复杂"。

- is、id、weakref 都是 Python 1.x 起就存在的 "引用模型" 文物：它们证明 "引用" 这个思想从第一版就刻进了语言核心。

- 类型标注姗姗来迟：函数注解 PEP 3107 (2006) 本来只是为未来多样化用法留接口；直到 PEP 484 (2015) 才确立类型标注语法与 typing 模块——本书作者至今对其持保留意见，认为它触碰了动态类型的价值根基。

- Python 3.12 引入的 "immortal object (不朽对象)" 是 CPython 对内存管理的又一次优化：把缓存对象标记为永不回收，配合 3.13 开始的 free-threaded 等实验，继续围绕 "共享引用 + 引用计数" 这个模型演进。

高级开发者需要掌握的相关知识

- CPython 内部结构：PyObject、obrefcnt、obtype；INCR/DECREF 宏；PyMemFree 与内存池 (pymalloc)。

- 分代 GC 细节：gc.getobjects()、gc.getreferrers() 做泄漏定位；gc.disable() 的代价与适用场景。

- 弱引用的全部 API：weakref.ref、WeakKeyDictionary、WeakValueDictionary、finalize (回调)、proxy，以及 "哪些类型支持弱引用" 的规则。

· sys.getrefcount、sys.intern、id 的实现内幕与"别依赖实现细节"的工程纪律。

· 现代同源话题：PEP 649 延迟注解求值、mypy/pyright 静态检查生态、@dataclass 的引用语义.....都能用本章模型一句话说清。

学习建议 (Learning Advice)

· 重要程度：★★★★★ (本教材最核心的一章，没有之一)

· 应该掌握到什么程度：

1. 复述级：能向别人解释"变量=贴纸、对象=物体、引用=贴纸到物体的线"；能说出"类型在对象上，不在名字上"。

2. 默写级：能不看材料判断——给定三段代码 (`B='Python'` / `B[0]='Python'` / `B=A[:]`)，正确说出 A 是否变化并解释原理。

3. 运用级：拿到任何 Python 代码，能画出关键变量的引用图；能嗅出"变量为何意外变化"的共享引用嫌疑。

4. 准则级：背下 `==` 比值、`is` 比身份 (单例用 `is`，其余用 `==`)；面对共享可变对象主动考虑拷贝；函数边界处警惕别名污染。

· 必须反复做的练习：

· 用 `id()` 观察 `a=3`；`b=a`、`a=(300,)`；`b=(300,)` 等组合的 `id` 是否相同，亲身体验缓存。

· 手写 `L2 = L1` 与 `L2 = L1[:]` 两套代码，各自 `L1[0]=...` 后打印两个列表，加深肌肉记忆。

- 提前预防：看到任何"两个名字指向"句法（赋值、传参、for 中的共享元素）都问一句"它俩是不是同一个对象？"
- 后续应该学习哪些相关内容：
- 第 7 章字符串：不可变对象的代表，继续巩固引用模型。
- 第 8 章列表与字典、第 9 章元组与文件：可变 vs 不可变的操作全貌。
- 第 14 章函数进阶：参数按引用传递 & 可变默认参数陷阱——"动态类型无处不在"在此落地为具体坑。
- 第 32 章多态与鸭子类型：把 "types live with objects" 变成你自己的设计武器。